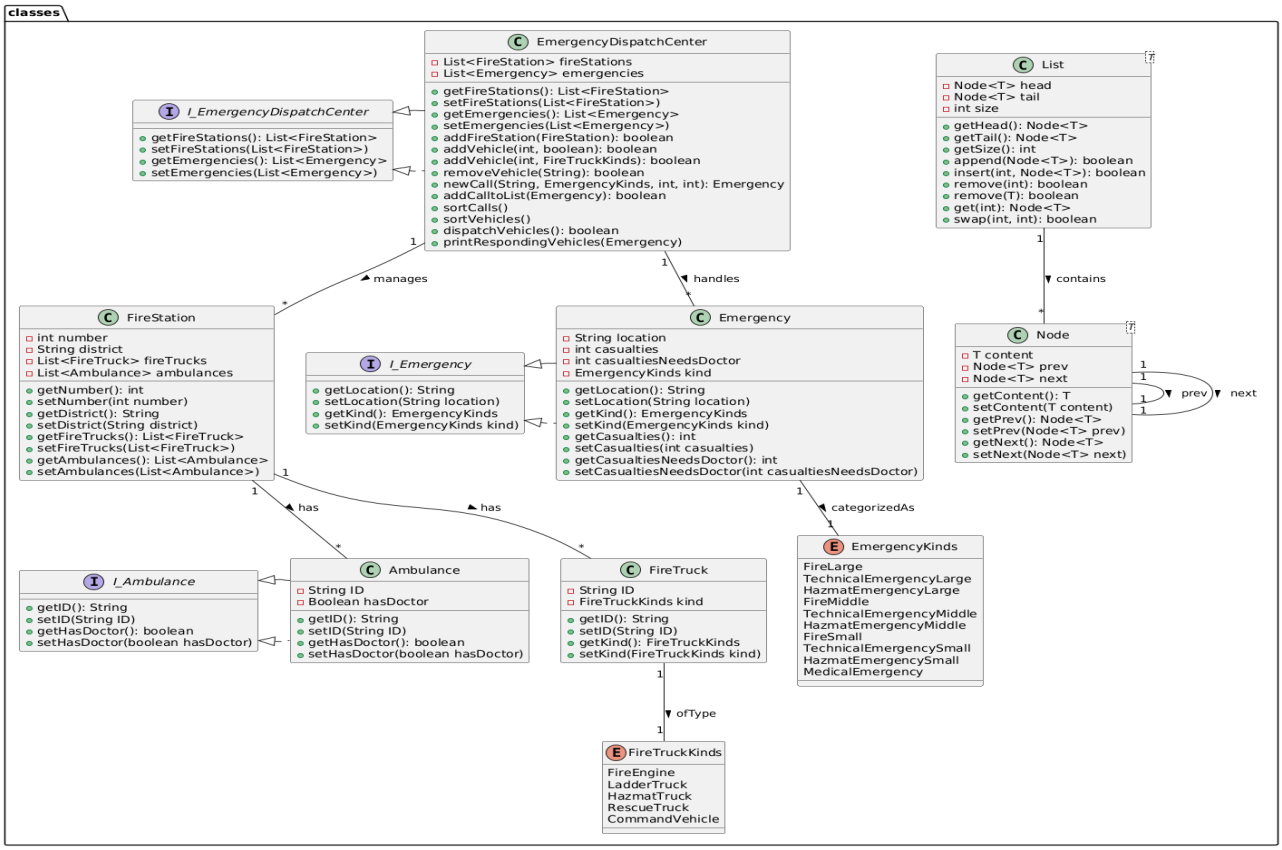
SEPB ASSIGNMENT

FEBRUARY 14, 2025

SAYOOJ KUNDANY SUREJ
753153

INTERACTION OF CLASSES



Emergency Dispatch Center class will act as a main class. It registers new fire stations and emergencies, by sorting them using priority using linked list. When an emergency is reported it creates a new Emergency instance using addCalltoList method. The dispatchVehicles method iterates over emergencies and allocates suitable vehicles.

The FireStation class holds the data for emergency vehicles, including ambulances and fire trucks. The list grows and shrinks dynamically when vehicles are added or removed.

Th Ambulance and FireTruck classes interact with fire stations. Both these classes creates unique IDs for the vehicles added based based on the requirements. The Ambulance class distinguishes between ambulances with or without a doctor. The FireTruck class uses the FireTruckKinds enumeration to specify the truck types.

Emergency class manages all the emergencies. It interacts with emergency dispatch center and fire stations. The class store the information such as location, severity and number of casualties. The type of emergency is categorized using the EmergencyKinds enums.

The dynamic nature of the system is done by the classes List and Node, which implement a doubly linked list structure. These classes manages the addition, removal and traversal through the list.

In summary, the interaction between EmergencyDispatchCenter, FireStation, Emergency, Ambulance and FireTruck classes forms enables to create an efficient emergency dispatch system. Node and List classes creates the dynamic nature if the system. Enumerations EmergencyKinds and FireTruckKinds supports the structure

CLASSES WITH ATTRIBUTES

1) **Ambulance class**
   a) String ID: A unique ID for ambulance which is essential for tracking the ambulance during allocation and dispatch
   b) Boolean hasDoctor: Decides whether the ambulance has a doctor or not. Used for allocating ambulances with doctors for patients who needs medical attention.
2) **FireTruck Class**
   a) String ID: A unique ID for the fire truck. Helps in tracking and managing fire trucks within the system.
   b) FireTruckKinds kind: An enumeration indicating which type of fire truck is used.
3) **FireStation Class**
   a) Int number: Unique ID for the fire station.
   b) String district: The name of the district the fire station serves.
   c) List<FireTruck> fireTrucks: List of fire trucks which is managed using a dynamic linked list.
   d) List<Ambulance> ambulances: List of ambulances available also managed dynamically using linked list.
4) **Emergency Class**
   a) String location: The location of the emergency.
   b) int casualties: The total number of casualties
   c) int casualtiesNeedsDoctor: The number of casualties that requires immediate medical attention.
   d) EmergencyKinds kind: An enumeration which indicating the type of emergency such as FireLarge or MedicalEmergency.
5) **EmergencyDispatchCenter Class**
   a) List<FireStation> fireStations: List of all registered fire stations in the system.
   b) List<Emergency> emergencies: A list of all the emergencies which are to be attended.
6) **Node Class**
   a) T content: The content that needs to be stored in a node.
   b) Node<T> next: Points to next node in the list
   c) Node<T> prev: Points to previous node in the list.
7) **List Class**
   a) Node<T> head: The starting point of the linked list
   b) Node<T> tail: The end point of the linked list.
   c) int size: The number of elements in the list.

Short explanation

- EmergencyDispatchCenter.dispatchForEmergency method(additional):
  This method determines the actual set of vehicles to dispatch for an emergency based on the emergency. The vehicles required for each kind of emergency is saved in a multi-dimensional array. It then checks for casualties that need doctor if needed. This calculates the total number of vehicles required for dispatch.

- List.swap method:
  The code swaps the two nodes by updating the previous and next nodes of both the nodes to point each other. Then, the previous and next pointers of nodeOne and nodeTwo are exchanged, making nodeOne take nodeTwo's place and vice versa. Finally, if either node was the head or tail of the list, the head or tail pointer is updated to reflect their new positions.

- EmergencyDispatchCenter.sortVehicles method:
  First sorting is done on the Firestation class for each station. This is done by separately sorting the list for firetrucks and ambulances by splitting their IDs. This is then called in the Emergency Dispatch Center and sorts for each firestation.

- EmergencyDispatchCenter.dispatchVehicles:
  This method checks if each emergency can be handled by the available fire stations. For each emergency in the list, it determines the required vehicles using the dispatchForEmergency method, then goes through all fire stations to find one in the same district as the emergency. Once it finds the matching fire station, it counts the number of available fire trucks of each required type; if the station doesn't have enough trucks for any requirement, the method returns false, indicating the emergency cannot be fully serviced. If all emergencies can be handled, the method returns true.